02 — Engineering & Operations Lessons

Cập nhật 2026-11-04 (branch *hmm-2nd-idea*). Phạm vi: lập trình, cấu hình môi trường, quản lý Kaggle, tổ chức thử nghiệm, lỗi đã gặp và cách sửa, bài học thành công / thất bại. Không chứa kết quả khoa học (→ *01*) hay kế hoạch (→ *03*). Bảng tài khoản và dataset: *ENVIRONMENT_SUMMARY.md*. Bản đầy đủ các vòng trước (ý tưởng 1, SelfCal, GPU L4): *history/02_ENGINEERING_OPS_NOTES_full_through_2026-11-03.md*.

1. Nguyên tắc tối ưu tài nguyên (theo `HMM_Navigation_Yeu_Cau_Nghien_Cuu_Cap_Nhat.docx`, đã kiểm chứng)

Tài nguyên	Thực tế	Quy tắc
Phiên CPU Kaggle	4 vCPU, ~30 GB RAM, 5 phiên / acc, không tốn hạn mức GPU	**Mọi việc của ý tưởng 2 chạy trên CPU** (đánh giá từ cache ~1 ms/frame; sinh cache `nav_demo --cache-only` cũng CPU)
Phiên GPU	2 / acc (T4 hoặc L4×4), 30 GPU-giờ / tuần / acc	Chỉ khi thật cần (trích descriptor mới, luật ý tưởng 1 SH/CST-JS); ước GPU-giờ trước, báo user nếu thiếu
Lấp phiên	—	Lấp hết phiên của một acc rồi mới sang acc kế (thứ tự `automation/active_accounts.txt`); `central_monitor.py` tự làm
Kẹt hàng đợi	—	Không có output 10 phút (T4) / 8 phút (L4, CPU) ⇒ xoá + đẩy lại, tối đa 5 lần, rồi soát lỗi code, rồi 2×T4 (monitor tự làm)
API Kaggle	Giới hạn theo IP (~80 tiến trình hỏi trạng thái ⇒ bị chặn hàng chục phút)	**Một** `central_monitor.py` cho mọi đợt (khởi động lại với cả thư mục cũ và mới; trạng thái trong `state.json` nên không mất gì)
Đĩa phiên chat	~30 GB, có hạn mức riêng	Kéo đầu ra chọn lọc (`--pull-pattern`); xoá bản cục bộ sau khi publish
Song song cục bộ	4 lõi	`OMP_NUM_THREADS` = lõi / số tiến trình; `xargs -P 4`

Luồng chuẩn của một đợt (đã dùng cho mọi đợt `hsi1`–`hsi20`): (1) smoke test cục bộ đầu-cuối trên 1–4 run thiết kế; (2) đóng gói LPT theo chi phí đo (`automation/hsi_wave.py`; job nặng như Boreas tách theo seed); (3) `fanout.py --groups --build-only` (tự đồng bộ code dataset mọi acc); (4) một monitor đẩy / kéo; (5) gom ngay khi từng kernel xong, kiểm `cache_path` trong JSON.

2. Công cụ (ý tưởng 2) — cách dùng

Việc	Lệnh
đánh giá biến thể trên cache	`analysis/hsi_eval.py <cache_dir|DIR:navcache_[@th■ m■c]> <ds> <desc> <seeds> --variants A,B --frames --tag t` → `results/hsi_<ds>_<desc>_s<seed>_<tag>.json` (+ `hsifr_*.npz` mặt nạ)
nhóm job một đợt	`automation/hsi_wave.py <groups.txt> <n_kernels> <variants> --tag t` (13 dataset-seed-group, nguồn `DIR:navcache_@hmm-navcache-v2` / `@hmm-e37-caches`)
dựng kernel	`automation/fanout.py <work> <prefix> <groups.txt> --groups --build-only --cpu [--nodumps] --ds a/b,c/d --par 4 --nocopy`

Việc	Lệnh
theo dõi	`automation/central_monitor.py <state.json> --gap 15 --run-poll 180 --nocopy [--pull-pattern '^(results\.tar|.*\.log)$'] <kernel_dirs...>`
gom kết quả	`automation/hsi_collect.py <out> <run_dir...> --tagre '_e8[.]'` (**bắt buộc** `--tagre`: kết quả chính, lỗ 10/50 % và tập mới trùng khoá dataset/descriptor/seed)
so sánh biến thể	`analysis/hsi_compare.py <dir> A,B,C [--base <oc dirs>] [--all] [--md out]` (mặc định chỉ run phát triển có SH; `--all` mọi run)
H1–H5 so với SH	`analysis/hsi_summary.py <dir> <oc dirs>` (báo cáo biến thể tên `HSI`; `paper/build_hsi_report.py` tự đổi tên)
tiêu chí đăng ký trước	`analysis/hsi_entry8_test.py <dir> HSI-VN HSI-VMN` (T1–T4 so với HSI-rot-m-sc)
quét α	`analysis/hsi_alpha.py <dir> --family HSI-VN HSI-VN --md --png`
sinh tập mới (bố cục lỗ mới)	job `hsi_fresh.py <ds> <desc> <seed0> <n> --keep navcache_rXX --variants oracle` (runner thêm thư mục dump) — chạy `nav_demo --cache --cache-only` rồi `hsi_eval`, giữ cache trong `/kaggle/working/navcache_rXX`
kéo file từ kernel nhiều acc	`automation/fetch_outputs.py <state.json> <dest> '<slug regex>' --pattern 'navcache_.*\.npz' --flatten` (token của chủ kernel)
publish dataset công khai	`automation/publish_dataset.py <dir> <slug> "<title 6–50>" "<subtitle 20–80>" --owner caotienminh1`
báo cáo	`paper/build_hsi_report.py <main dir> <oc dirs> --cmp name=dir[@v1,v2] ... --confirm <dir>` (tên `*_ho` = tập con có SH)
hình	`analysis/hsi_lockfig.py`, `analysis/hsi_alpha.py --png` (matplotlib phải `pip install` trong phiên)

3. Tổ chức thử nghiệm

- **Đăng ký trước mọi quyết định:** viết tiêu chí + quy tắc quyết định vào `RESEARCH_LOG.md`, commit + push TRƯỚC khi kernel kết quả xong; giờ ghi theo `date -u` của hệ thống (đã có lần ghi lệch 10 phút — đã sửa).
- **Phân chia dữ liệu:** thiết kế (seed 0 × 4 descriptor, phân tích cục bộ tuỳ ý) → phát triển (seed 0–9, chẩn đoán được) → **tập kiểm tra mới cho mỗi quyết định**. Đã dùng: seed 10–12, 13–15, 16–18. **Kho dự trữ seed 19–27** (`caotienminh1/hmm-navcache-r19`): rút một nhóm 3 seed chưa dùng cho mỗi quyết định, đánh dấu "đã dùng" trong `03_HANDOFF.md`. Lý do không dùng lại một tập: quá khớp thích nghi đo được (lợi ích +0.014 trên tập giữ ra dùng 6 lần co còn +0.007 trên tập mới).
- **Sinh cache một lần, publish, mount nhiều lần:** sinh 3 seed × 36 run mất ~1 giờ trên 18–25 phiên CPU (Boreas ~42 phút / seed là đường găng ⇒ mỗi seed Boreas một job); đánh giá một tập đã publish chỉ ~15 phút (6 kernel). Kernel sinh cache là private ⇒ kernel acc khác không mount được ⇒ phải kéo về + publish dataset công khai (không token, không dữ liệu nhạy cảm).
- **Đo trước khi chạy lớn:** mọi ý tưởng mới được đo trên tập thiết kế / vài run phát triển cục bộ (vài phút) trước khi dựng đợt Kaggle; 6 ý tưởng bị loại ở bước này (xem `01` §4).
- **Một cờ cho mỗi thay đổi** trong `online_v2` (mặc định = hành vi cũ) và một biến thể có tên trong `VARIANTS` ⇒ ablation và tái lập dễ; kiểm hồi quy: biến thể cũ cho đúng số cũ sau mỗi refactor.

4. Lỗi đã gặp và cách sửa (ý tưởng 2)

Lỗi	Triệu chứng	Sửa
runner `--nodumps`	FileNotFoundError khi in thư mục dump, mọi kernel hỏng trong 1 phút	`os.makedirs(dump_all, exist_ok=True)`
nhiều dataset cache cùng tên file	đợt Entry 4 mount 4 dataset trùng tên → đọc nhầm cache lỗ	token tường minh `DIR:navcache_@<th■ m■c>`; JSON ghi `cache_path`; xoá + dựng lại đợt
gom trộn kết quả	holes và main trùng khoá → ghi đè biến thể	`hsi_collect.py --tagre`
OpenMP quá tải	2 tiến trình GBM chậm ~10×	`OMP_NUM_THREADS`
băm đơn vị hiệu chuẩn	`(unit*k) % 2` cho mọi đơn vị CMU vào hiệu chuẩn	`calib_units` ngẫu nhiên tất định
cột toàn NaN	HistGradientBoosting "window shape cannot be larger"	`Model` bỏ cột toàn NaN
bảng báo cáo rỗng	biến thể chưa tồn tại trên tập cũ → 0 run	danh sách biến thể riêng cho từng bảng (`--cmp name=dir@v1,v2`)
tải nhầm cả dataset	`kaggle datasets download -f ""` (biến rỗng) kéo `archive.zip` 1.3 GB	luôn kiểm tên file trước; liệt kê bằng `kaggle datasets files`
lệnh `mkdir` sai thư mục	tạo thư mục rỗng trong repo, `rm -rf` xoá nhầm file vừa tải	dùng đường dẫn tuyệt đối, không chuỗi `cd` + lệnh xoá
`kaggle datasets create` trả 499	dataset vẫn tạo xong	kiểm bằng `datasets status` + đếm `datasets files`
worker phiên chat khởi động lại	monitor chết, kernel vẫn chạy trên Kaggle	khởi động lại monitor với `--pull-first "<regex>"` để kéo đầu ra đã xong
heredoc `'EOF'` + `\\n`	nhãn hình hiện "\n"	trong heredoc có nháy, viết `\n` một lần
background `until` + timeout 600 s	lệnh chờ bị chuyển nền rồi dừng	dùng `run_in_background` với `timeout` dài, hoặc Monitor `tail -F | grep`

5. Bài học thành công / thất bại

- **Thành công:** (1) đo cơ chế trên dữ liệu thiết kế trước (chẩn đoán vách khoá, hubness, đồng dạng) tiết kiệm hàng giờ Kaggle; (2) cache công khai + CPU ⇒ một đợt 441 run × 4–10 biến thể chỉ 15–40 phút, không tốn GPU; (3) đăng ký trước + tập mới ⇒ kết luận đứng vững (HSI-VN đạt 4/4 trên seed 16–18); (4) một monitor + `--pull-pattern` ⇒ không bị chặn API suốt phiên, không kéo file lớn.
- **Thất bại / cần tránh:** (1) tập thiết kế 28 run không lộ cái giá điều hướng của Entry 6 ⇒ luôn xem sai số điều hướng và đuôi p95, không chỉ recall; (2) tiêu chí cơ chế quá chặt (T3 Entry 8) làm một luật trội Pareto chưa được nhận — đặt tiêu chí theo thước đo kết quả, tiêu chí cơ chế để báo cáo; (3) giả định "lỗi điều hướng do chụm sai ngắn" sai — đo trước khi viết code.
- **Bảo mật:** token Kaggle chỉ ở `~/.kaggle/tok_<acc>` (chmod 600), không bao giờ trong repo / dataset / báo cáo / prompt; repo là public.

6. Entry 10 (OWF, 2026-11-04) — vận hành và bài học

Việc	Cách / số đo
một run OWF	`python3 analysis/owf.py <cache_dir> <dump_dir> <ds> <desc> <seed> --out r.json --frames fr` (cần cả navcache — lỗ, odometry, nhãn — và dump descriptor; ba bộ lọc q chạy trong một tiến trình)

Việc	Cách / số đo
đợt Kaggle	`automation/owf_wave.py <groups> --set main|abl|holes|r19|r19hsi --kernels N` → `VPR_CODE_DS=dive-vpr-owf fanout.py ... --groups --cpu --build-only --par 4 --ds <caches>` (`--nodumps` cho `r19hsi`) → `VPR_CODE_DS=dive-vpr-owf central_monitor.py <state> --gap 20 --run-poll 300 --nocopy <dirs>` → `owf_collect.py <work> <out> [--frames]` → `owf_summary.py`
thời gian	cục bộ 2–14 ms/frame (3 bộ lọc, 1 lõi); phiên CPU Kaggle (4 job / 4 vCPU) 6–36 ms/frame, chậm ~2.3×; Boreas 8448-d ~33 phút / run ⇒ tập A 404 run trên 30 kernel CPU ~85 phút đồng hồ, tập B 108 run trên 10 kernel ~50 phút; 62 kernel tổng, 0 job lỗi
bộ nhớ	≤ 4 GB / job (Boreas 8448-d), 4 job / phiên CPU 30 GB ổn

Lỗi / sự cố và cách sửa. - **Checkout cũ:** container của phiên bắt đầu ở commit tạo branch (`7502e2c`), không có Entry 1–9 của HSI; chỉ phát hiện khi push bị từ chối. Bài học: `git fetch` + `git log HEAD..origin/<branch>` NGAY khi bắt đầu, và trước mỗi push. - **Hai phiên chat song song cùng branch / cùng acc:** (i) kernel thăm dò của phiên này đã version `<acc>/dive-vpr-analysis` bằng code cũ (thiếu `hsi*.py`) ⇒ có thể làm hỏng kernel mới của phiên kia; đã đồng bộ lại bằng HEAD và thêm `VPR_CODE_DS` (code dataset riêng mỗi phiên, cả `make_eval_kernel.py`, `fanout.py`, `central_monitor.py`); (ii) cả hai cùng publish `hmm-navcache-r19` trong cùng phút (lỗi "title already in use" ở phiên này — dataset của phiên kia, đủ 324 file); (iii) tập quyết định chuyển sang nhóm dự trữ cuối (25–27) để không giành nhóm 19–21. - **Thử nghiệm sai:** `opt()` đọc `sys.argv` sau khi đã cắt cho `import nav_demo` ⇒ cờ `--frames` / `--out` mất (sửa: đọc trước khi cắt); `rmdir $d/$(...)` bị chặn bởi kiểm an toàn ⇒ dùng đường dẫn tương đối tường minh; `tar --exclude` phải đặt trước đường dẫn. - **GitHub 500 khi push:** thử lại với lùi luỹ thừa thành công ở lần 2.

Bài học. (1) Đo khoảng trống trước khi chọn ý tưởng (MAP chuỗi đúng vs recall báo) chỉ ra nút thắt nằm ở cấu trúc trạng thái, không ở lớp quyết định. (2) Một vòng thiết kế cục bộ trên seed 0 (CPU 4 lõi, ~2 giờ) đủ để chốt cấu hình; mọi tham số chọn bằng nguyên tắc (đơn vị excl, evidence trực tuyến) và ghi lại đường đi thiết kế. (3) Sai số điều hướng phải được đo ngay trong thiết kế: tập thiết kế cũ chỉ có Boreas có err, nên điểm yếu điều hướng chỉ lộ ra ở tập giữ ra.

7. AUTOLOOP (Entry 11–15, 2026-11-04 14:15 → 02:15 UTC) — vận hành và bài học

Việc	Cách / số đo
kho seed mới	`owf_wave.py <groups> --set fresh:49-54` (`hsi_fresh.py --keep navcache_r49`, kèm HSI-VN / VMN / oracle) → 34 kernel CPU, ~2.5 giờ → `fetch_done.py <state> <dest> '^fr49-' --pattern '^navcache_.*\.npz$'` (kéo dần cache của kernel đã xong) → `publish_dataset.py <dir> hmm-navcache-r49 ... --owner synchan5704` (lỗi "499 Client Error" vô hại; đếm file bằng `kaggle datasets files ... --page-size 200 --page-token`)
đợt kiểm tra	`owf_wave.py <groups> --set fresh_owf:49-51 --pool r49 [--bank | --banks "S:0.97,1.0,1.03;S5:0.94,...,1.06" --noplain] --hsiframes --modes xms0.9_16:HSI-VN --kernels 32` → `fanout.py ... --groups --cpu --build-only --par 4 --nocopy --ds synchan5704/hmm-navcache-r49,<oxford>,<stlucia>`; mỗi job `owf_hsi_job.py` = OWF → OWF-H (+ mặt nạ `hsifr_*`) trong cùng kernel; 288 job / 32 kernel CPU ~1.3 giờ (ngân hàng 6 bộ lọc), ~2.5 giờ (6 + 10 bộ lọc)

Việc	Cách / số đo
quyết định	gom `results/` của `<slug>_out` → `owf_union.py <fr> <hfr> <navcache> out.json --tag <tag>xms09_16 --ftag _<tag> --variants "H|m(1e-5,16)|m(1e-8,2)|t(3e-4,0.5)"` → `autoloop_board.py --group 49-51 --rule NAME=kind:path@variant ...` (bảng Score V1/V2/N1) + bootstrap theo run
phiên L4	`--l4 --par 46` (L4 có ~48 vCPU): 48–50 job / kernel chạy song song một lượt; NHƯNG L4 tốn quota GPU tuần (30 giờ / acc): hết ở synchan5704, quynhle07, nguyenhoa99, caotienminh1 trong ngày 2026-11-04; yeshenhi không có L4 (kernel L4 rơi về máy yếu, 50 job chạy > 3 giờ)

Lỗi / sự cố. - `md2pdf.py` nhận ĐÚNG hai đối số `<in.md> <out.pdf>`; gọi với ba file `.md` đã ghi đè `01_SCIENTIFIC_FINDINGS.md` bằng PDF (đã khôi phục từ commit trước). Luôn gọi từng file với đường ra `.pdf`. - `pkill -f "<chu■i>"` giết luôn shell đang chạy lệnh (dòng lệnh của shell chứa chuỗi đó) ⇒ dừng monitor bằng PID (`ps aux | grep "[c]entral_monitor"`). - Vòng chờ grep "error|failed" khớp dòng vô hại "GPU quota spent -> next account" ⇒ chỉ grep `NEEDS_FIX|STALL_GIVEUP|rc!=0: [1-9]` trên log hiện tại. - Biến vòng lặp `b` trong `owf_wave.py --banks` che biến `b` (seed cuối) ⇒ đổi tên; tag phân tích `split("_s")` sai với salad / stlucia ⇒ `rpartition("_s")`. - Thời điểm ghi trong nhật ký phải lấy từ `date -u` / thời điểm commit, không ước lượng.

Bài học. (1) Đánh giá ngoại tuyến từ khung đã lưu (`fr_*_b1.npz` + mặt nạ HSI `hsifr_*`) cho phép thử hàng chục luật quyết định trong vài phút; chỉ đổi BỘ ƯỚC LƯỢNG mới cần chạy lại trên Kaggle. (2) Chẩn đoán oracle (chuỗi đúng ở đâu, báo thiếu ở đâu, theo quãng đường từ đầu chuỗi) chỉ ra ngay nút thắt (CMU đầu chuỗi; countryside tỉ lệ VIO). (3) Chạy luật đương nhiệm và luật mới trong CÙNG đợt, trên CÙNG run ⇒ so sánh cặp có CI hẹp (±0.002–0.006).

8. Phiên 2026-11-05 (chiều): engine GPU, đợt L4 thật sự dùng 4 GPU, kho r55

Việc	Cách / số đo
tài khoản	`namhunhh`, `duckzuykobip`, `dinhthinh` (token `~/.kaggle/tok_<username>`); `active_accounts.txt` chỉ còn ba tài khoản này. `namhunhh` có kernel của người dùng khác chạy song song: monitor tự chuyển sang tài khoản kế khi gặp "no free cpu session"
thăm dò máy	kernel nhỏ `machine_shape=NvidiaL4` in `nvidia-smi`, `os.cpu_count()`, `free -g` rồi xoá: 4 × L4 23 GB, 48 vCPU, 188 GB RAM, 20 GB đĩa
OWF trên GPU	`owf.py ... --engine torch`: cả ngân hàng q × tỉ lệ (× β nếu `--bbank`) trong một tensor float64. L4: 2.7–3.4 ms/frame cho 10 bộ lọc (numpy cùng máy 29 ms/frame); Boreas 76k frame 4.6 phút. Kiểm tương đương: `owf.py` hai engine trên cùng run + so `fr_*_b1.npz` (vị trí báo, quyết định ở 1−0.2 … 1−1e-8, đường cong vận hành)
đợt luật đương nhiệm	`owf_wave.py <groups> --set inc:setA+holes+natural --kernels 6 --par 24` → `fanout.py ... --groups --build-only --l4 --par 24 --nocopy --ds <navcache-v2, e37-caches, nclt/oxford/stlucia dumps, navcache-holes, natural-cache>` → một `central_monitor.py`. 657 run trong 35 phút trên 6 phiên L4, 0 lỗi (OWF trên 4 GPU, HSI trên 48 CPU của cùng phiên; 6 job / GPU)
nhóm seed mới + SOTA	`owf_wave.py <groups> --set inc:fresh55-57 --pool r55 --sota`: mỗi run một job `owf_hsi_job.py` (OWF GPU → OWF-H) và một job `sota_job.py` (top-100 trên GPU → TAO'26, Xu'21, Claxton'24, top-1, oracle)

Việc	Cách / số đo
quyết định ngoại tuyến	`owf_union.py` (biến thể bậc), `owf_hcal.py` (ngưỡng conformal theo tầng trên null hồi cứu tự tính lại từ chuỗi tô-pô, < 1 s / run), `hcal_board.py` (bảng theo dataset + chênh cặp với CI bootstrap). Tập A 399 run: 2.5–7 phút trên 4 lõi
kho seed mới	`owf_wave.py --set fresh:55-60` (288 job `hsi_fresh.py --keep navcache_r55`, 15 kernel CPU, ước 139 phút / kernel; thực tế 73–100 phút); chuỗi nền `runs/r55_chain.sh`: chờ đủ 15 kernel → `fetch_done.py` → `publish_dataset.py ... hmm-navcache-r55 --owner namhunhh`

Bài học. - Container phiên chat bắt đầu ở commit cũ (`7502e2c`) dù remote đã đi trước 50 commit với lịch sử khác (orphan). Phải `git fetch && git log HEAD..origin/<branch>` ngay đầu phiên, và `git reset --hard origin/<branch>` khi local không có gì riêng. - Đo trước khi chạy lớn tiếp tục cứu thời gian. Ý tưởng hiệu chuẩn conformal (Entry 16) và β-bank đều bị loại trên dữ liệu phát triển (399 run / 1 run) trước khi đụng tới nhóm seed mới. - Thời điểm ghi trong nhật ký phải lấy từ `date -u` ngay lúc viết (hai lần ước lượng sai 10–17 phút đã phải sửa). - Tập tổng quát (lỗ 10/50 %, off-map tự nhiên) được chạy chung đợt với tập phát triển nhưng gom vào thư mục riêng và chỉ đọc sau khi đã quyết định trên tập A, để chúng vẫn là bằng chứng không bị dùng để chỉnh luật.